

SOFTWARE FELLOWSHIP · DAY 4

Databases, Authentication & Real Data

Making our data survive a restart — and connecting the backend to a real database.

FastAPI

SQLAlchemy

SQLite

PostgreSQL

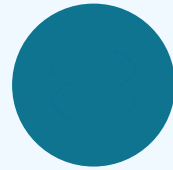
Instructor-led session · Lecture + Lab

What we'll cover



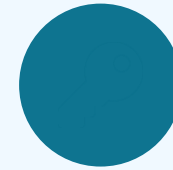
Why databases? 01

The problem they solve



SQL vs NoSQL 02

Two ways to store data



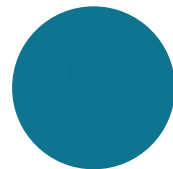
Keys & tables 03

Primary and foreign keys



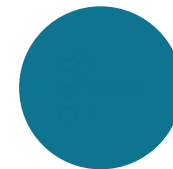
SQL basics 04

PostgreSQL / SQLite



The ORM 05

Talk to the DB in Python



CRUD 06

Create · Read · Update ·
Delete

Lecture first → then we build it live in the lab.

Right now, our data has amnesia.



Data in a Python list

(what we did on Day 3)

```
projects = []    # lives in RAM
projects.append({...})
```

Server restarts → **everything gone.**

Crash, deploy, power cut — same result. Nothing survives.



Data in a database

(what we build today)

```
db.add(project)
db.commit()    # written to disk
```

Server restarts → **data still there.**

Stored on disk, safe across restarts, shared by everyone.

What a database actually gives you



Persistence

Data lives on disk, not RAM — survives restarts, crashes, deploys.



Concurrency

Many users read and write at once, safely, without corrupting data.



Querying

Ask precise questions: “all shipped projects by this fellow” — instantly.

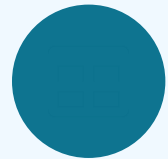


Integrity

Rules the data must obey: no duplicate emails, no orphan records.

A database isn't just storage — it's storage with rules, speed, and safety built in.

SQL vs NoSQL



SQL — Relational

Tables with rows & columns. Fixed schema.

- ▶ Data is structured & related
- ▶ Strong consistency & integrity
- ▶ Powerful queries with JOINS
- ▶ Postgres, MySQL, SQLite



NoSQL — Non-relational

Documents, key-value, graphs. Flexible schema.

- ▶ Data is fluid or unstructured
- ▶ Scales horizontally with ease
- ▶ Fast for simple lookups
- ▶ MongoDB, Redis, Firebase



Which today? SQL — our data is relational (fellows own projects), and it's the sensible default until you have a reason not to.

Tables, primary keys & foreign keys

fellows

id	name	email
1	Aarya	aarya@...
2	Bibek	bibek@...

id is the PRIMARY KEY — unique per row

projects

id	title	owner_id
1	Weather App	1
2	Bus Finder	1
3	Momo Map	2

owner_id is a FOREIGN KEY → must match a fellows.id



The foreign key is the glue.

It answers “whose project is this?” by pointing each project back to exactly one fellow — and the database refuses any owner_id that doesn't exist. No orphan rows.

SQL — four commands do 90% of the work

INSERT*— add a row*

```
INSERT INTO projects (title, owner_id)
VALUES ('Weather App', 1);
```

SELECT*— read rows*

```
SELECT * FROM projects
WHERE owner_id = 1;
```

UPDATE*— change a row*

```
UPDATE projects SET status = 'shipped'
WHERE id = 3;
```

DELETE*— remove a row*

```
DELETE FROM projects
WHERE id = 3;
```

Always pair UPDATE and DELETE with a **WHERE** — forget it and you change every row.

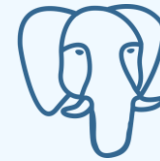
SQLite today, PostgreSQL in production



SQLite

A single file on disk

- ✓ Zero setup — no server to run
- ✓ The whole DB is one .db file
- ✓ Perfect for learning & prototypes
- ✓ Ships in every phone & browser



PostgreSQL

A real database server

- ✓ Runs as a service, many clients
- ✓ Handles heavy concurrent load
- ✓ Rich types, indexes, extensions
- ✓ The production default

The trick: with an ORM, switching between them is a one-line change. Same code, different engine.

The ORM: write Python, not SQL

Object–Relational Mapper — it translates between Python objects and database rows, both ways.



You write Python

```
project = Project(  
    title="Weather App",  
    owner_id=1,  
)  
db.add(project)  
db.commit()
```



SQLAlchemy

the ORM

translates ↔ both ways

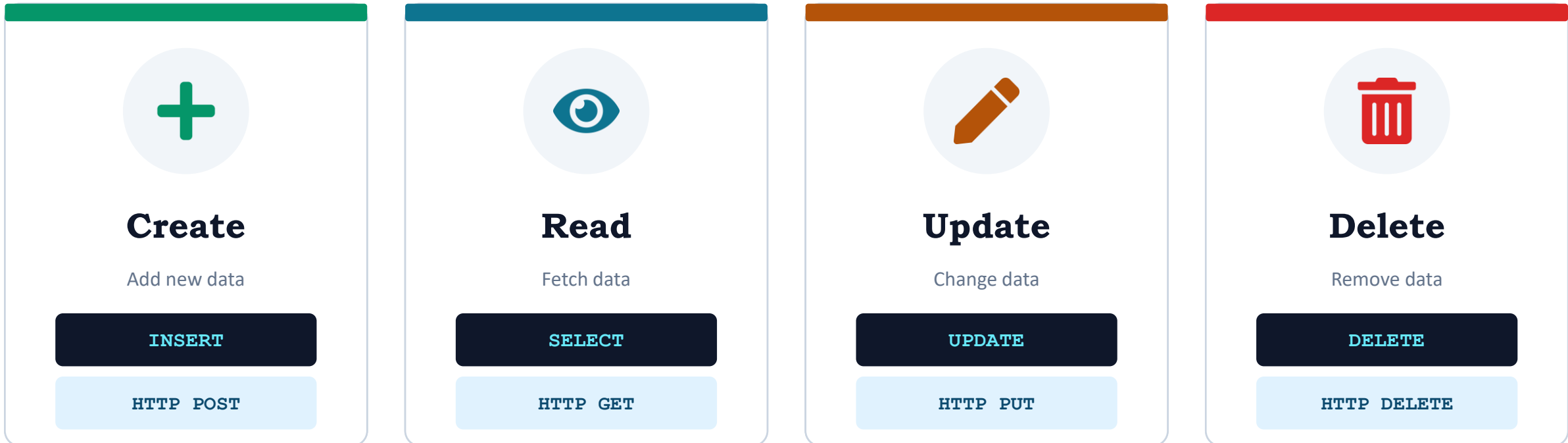


It runs SQL

```
INSERT INTO projects  
    (title, owner_id)  
VALUES  
    ('Weather App', 1);
```

Why bother? Fewer bugs, no hand-written SQL strings, and you can swap SQLite → Postgres without rewriting a line.

CRUD — the verbs of every app



***One idea, three names:** the CRUD verb, the SQL it becomes, and the HTTP method the client calls. Learn the row and you know all three.*

A login is just a row — with one rule.



Never do this

```
password = "hunter2"  
# stored as plain text  X
```

If the database ever leaks, every password is exposed — and people reuse passwords everywhere.



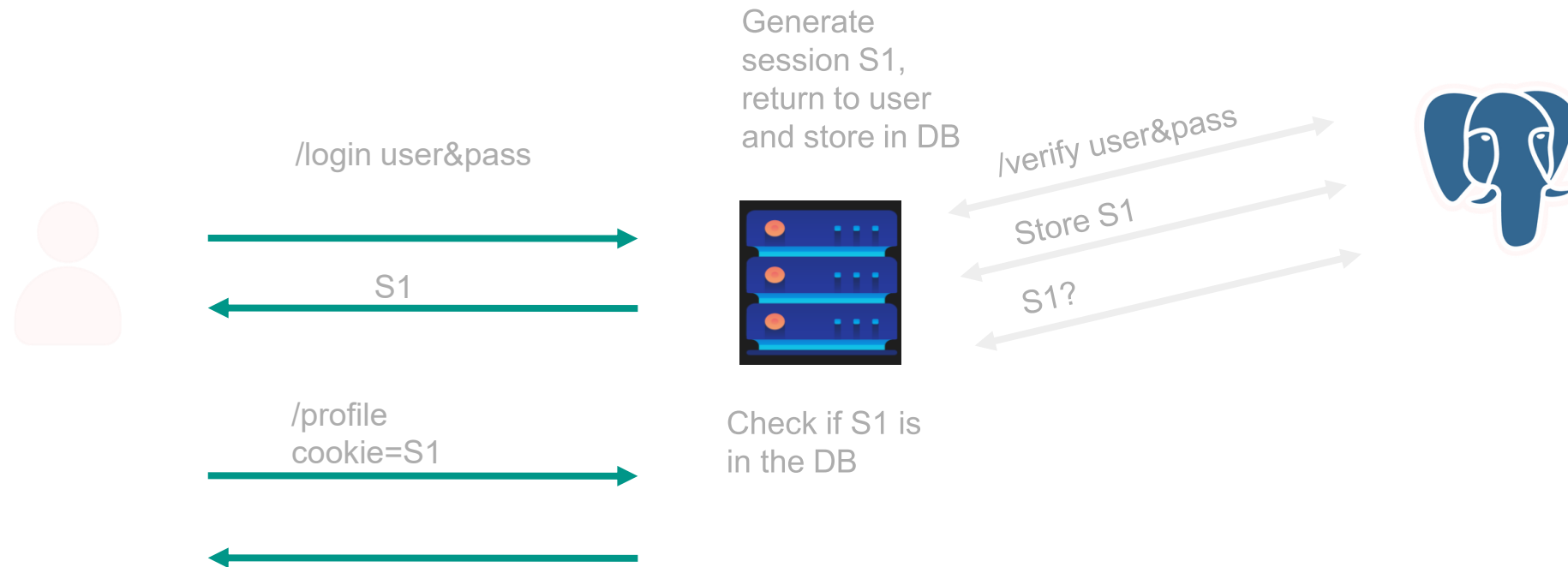
Do this — hash it

```
hash = bcrypt.hashpw(pw)  
# $2b$12$Zv787dZ...  ✓
```

Store a one-way hash. To log in, hash the attempt and compare. The real password is never saved.

We'll wire this up as the lab's stretch goal — it's the first step toward real login systems.

Stateless Backend



Backend App is stateless but entire system still stateful (database dies we lose everything)

The full request, end to end



...and back again.

The database returns the row, SQLAlchemy hands back a Python object, Pydantic (ProjectOut) shapes the JSON response, and FastAPI sends it to the client. Every endpoint you build today follows this exact loop.

HANDS-ON LAB

Expense-tracker API with a real database

Connect FastAPI to a database, build full CRUD, save real expenses, and push to GitHub.

FastAPI

SQLAlchemy

SQLite

GitHub

A Expense API — one table, full CRUD

Build an API that stores expenses in a real database. Add, view and delete them — and they survive a server restart.

[todo-app/](#)

The five endpoints:

POST

/expenses

GET

/expenses

DELETE

/expenses/{id}

```
expense-tracker/  
├── 1-sqlite/           # raw sqlite3 version  
│   ├── main.py  
│   ├── requirements.txt  
│   └── .gitignore  
├── 2-orm/              # SQLAlchemy ORM version  
│   ├── main.py  
│   ├── requirements.txt  
│   └── .gitignore  
└── README.md
```

Lab steps

1

Set up

`venv + pip install fastapi uvicorn sqlalchemy`

2

Build main.py

`connect → table → schemas → routes`

3

Run it

`uvicorn main:app --reload → open /docs`

4

Try all five

`add · view · tick off · delete your todos`

5

Prove it

`restart the server — your expenses are still there`

6

Push to GitHub

`git init → commit → push (not the .db!)`

Kill the server. Start it again.

Then GET /expenses.

```
$ curl localhost:8000/expenses  
[  
  {"id":1,"name":"Coffee","amount":5},  
]  
# still there. ✓
```



This is the whole point.

Your data now lives independently of your code.

SQL injection Live Demo

Demo website : <https://demo.testfire.net/index.jsp>

```
SELECT * FROM users WHERE  
username='admin' AND  
password='password123'
```

```
SELECT * FROM users WHERE  
username='admin' OR '1'='1' AND  
password='password123'
```

```
SELECT * FROM users WHERE  
username='admin' --' AND  
password='password123'
```

What you can do now

- ✓ Explain why apps need databases
- ✓ Design tables with primary & foreign keys
- ✓ Use an ORM to run full CRUD from Python
- ✓ Tell SQL and NoSQL apart — and pick one
- ✓ Read & write basic SQL
- ✓ Persist real data and push clean code to GitHub



Homework

Implement it using PostgreSQL, since in development you will not be using SQLite.

DAY 4 COMPLETE

Your data now survives.

Questions? Let's debug together.



Push your repo — we'll review a few live.